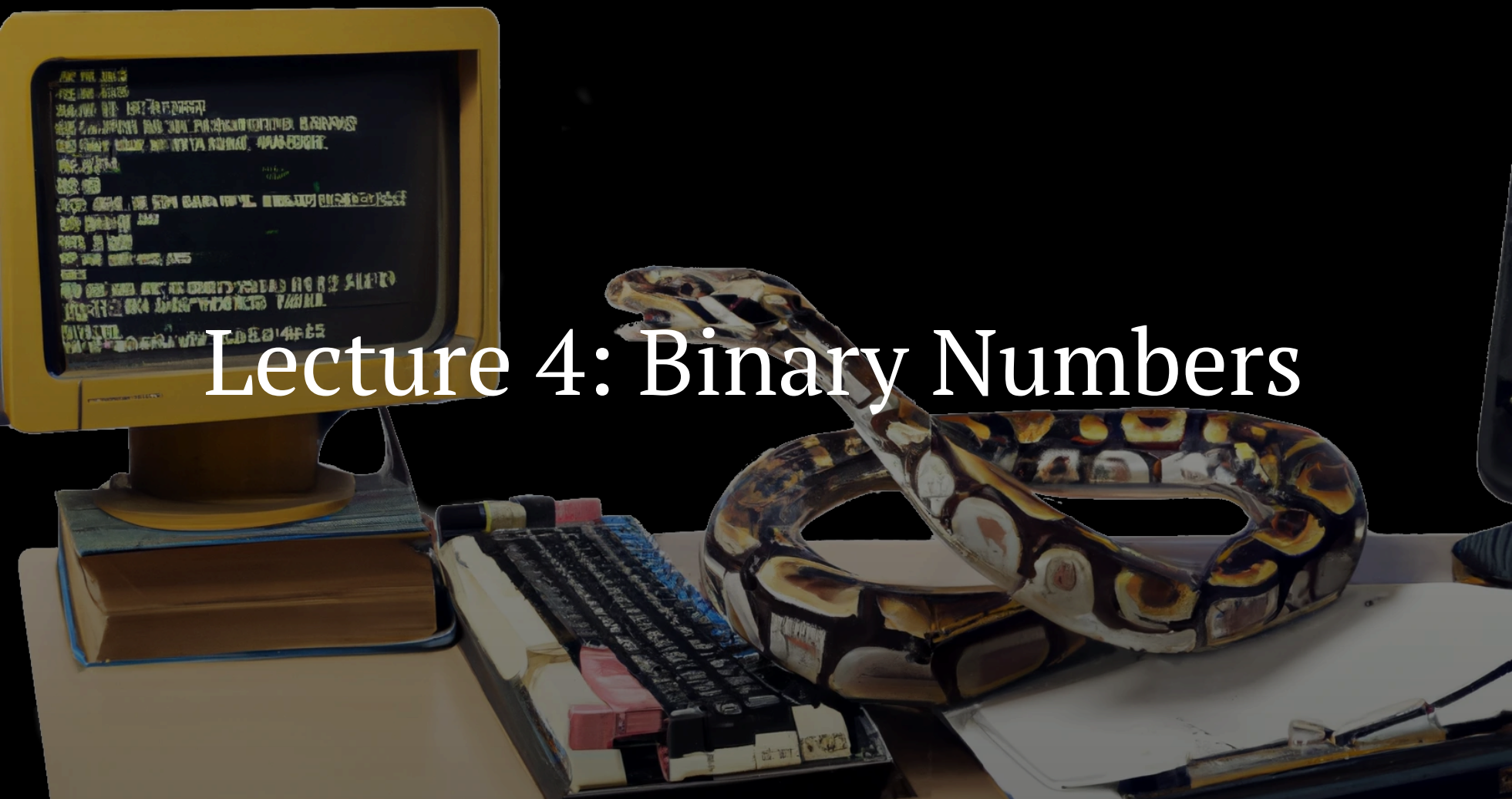




Lecture 4: Binary Numbers

Roadmap

- Review last class
 - Transistors
 - Digital logic
- Review bonus assignment
- Binary
- Exercise: Adder
- Computer Architecture
- Hexadecimal
- Binary Numbers in Python
- Homework Assignment

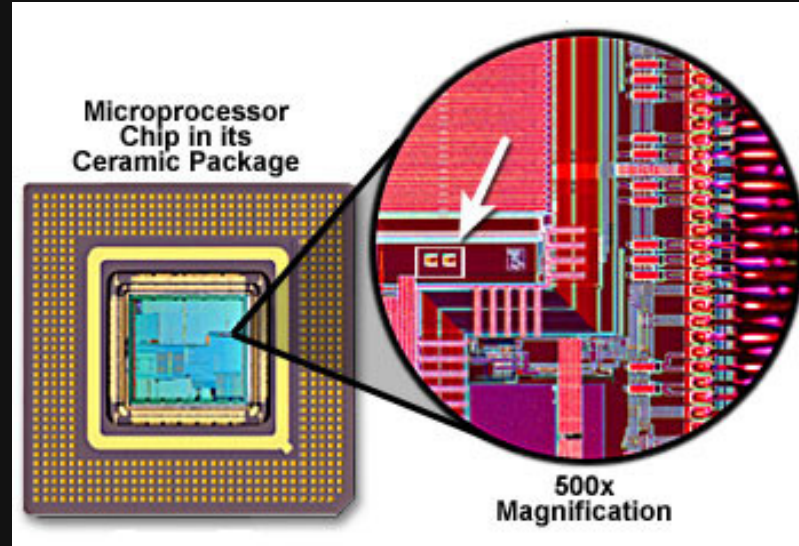


Transistors

- What is a transistor?
- Semiconductor device used to amplify or switch electrical signals
- Proposed in 1925
- First working device 1947
 - Nobel Prize (Physics) 1956
- Made from very pure silicon
- Can be used to make complex logic circuits



Transistors in Computer Chips





Moore's Law

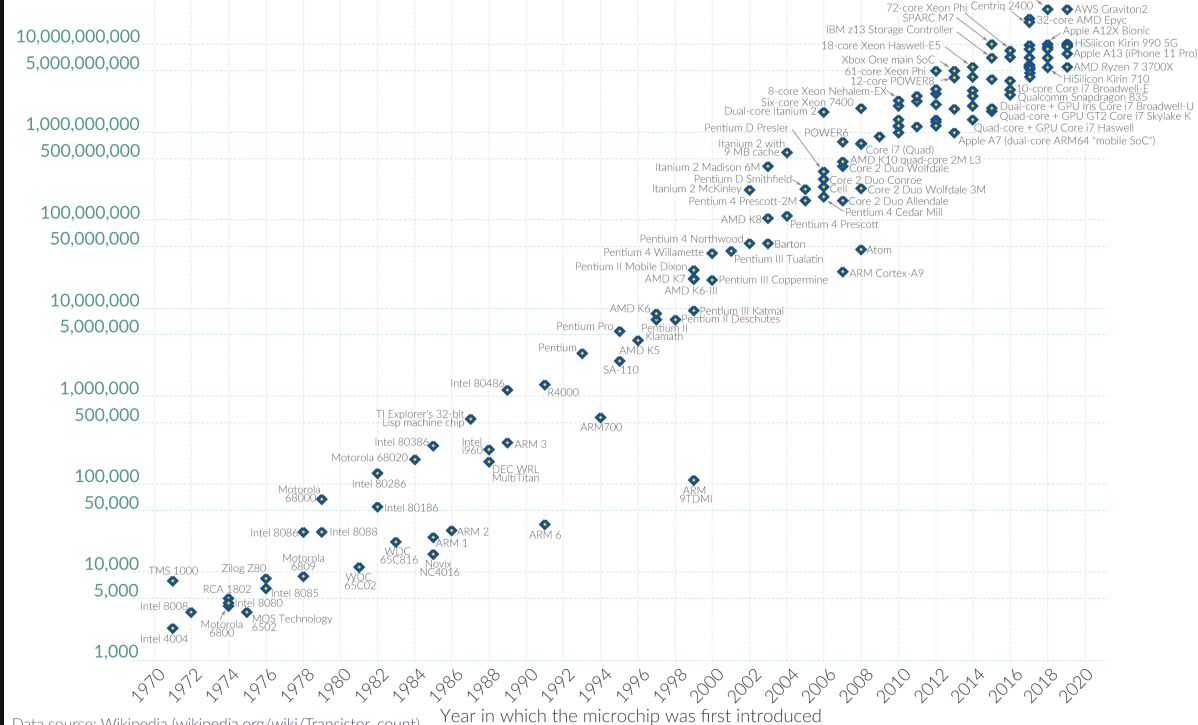
Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count

50,000,000,000



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under [CC-BY](#) by the authors Hannah Ritchie and Max Roser.

Logic Gates

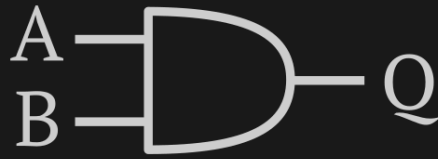
- Built from transistors acting as switches
- Implement Boolean logic
- Voltages indicate Boolean value
 - False: low logic level, usually 0 V
 - True: high logic level, usually 5 V, 3.3 V, 3 V, 1.8 V, 0.9 V
- Simulators:
 - <https://logic.ly/demo>
 - <https://circuitverse.org/simulator>

NOT Gate



A	Q
0	1
1	0

AND Gate



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1



OR Gate



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

NAND Gate



A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

XNOR Gate



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1



Exercise: 3-Way Vote

- Three toggle switches used for voting
- Light turns on if at least two switches are on
- Try it:
 - <https://logic.ly/demo>



3-Way Vote: Truth Tables

A	B	C	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		A	
B	C	0	1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1



3-Way Vote: Algebraic Notation

A	B	C	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		A	
B	C	0	1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

- NOT: $\overline{A}$
- AND: $A \cdot B$
- OR: $A + B$

$$Q = \overline{A} \cdot (B \cdot C) + A \cdot (B + C)$$

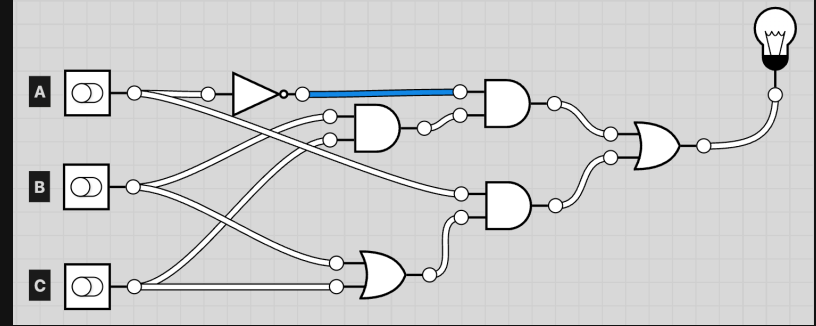


3-Way Vote: Solution

A	B	C	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		A	
B	C	0	1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

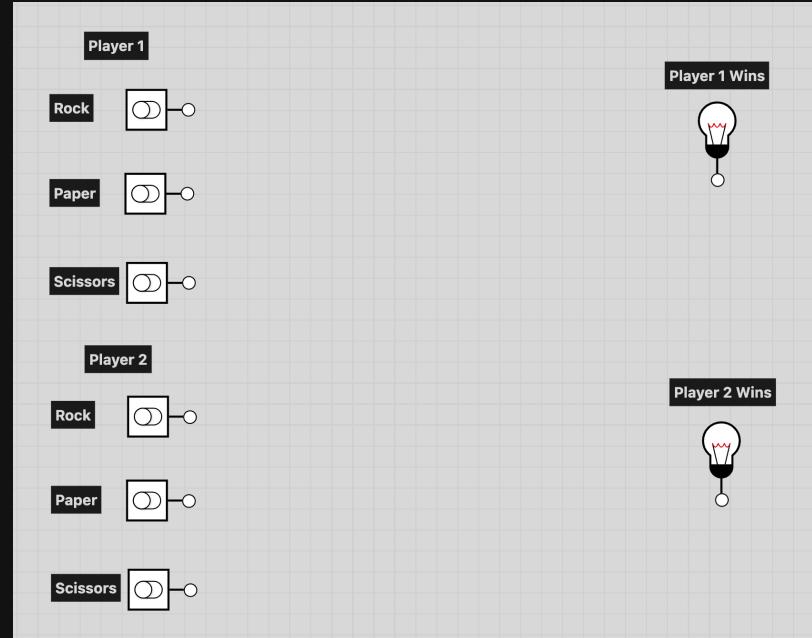
$$Q = \overline{A} \cdot (B \cdot C) + A \cdot (B + C)$$





Bonus Assignment: Rock, Paper, Scissors

- Bonus assignment: rock, paper, scissors game
 - Using digital logic on <https://logic.ly/demo>
 - Two players
 - Three switches per player
 - Two lights
 - If player 1 wins: light 1 on, light 2 off
 - If player 2 wins: light 1 off, light 2 on
 - Tips
 - Make truth tables for light 1
 - Write algebraic expression for light 1
 - Light 2 should be easy to figure out once you get light 1



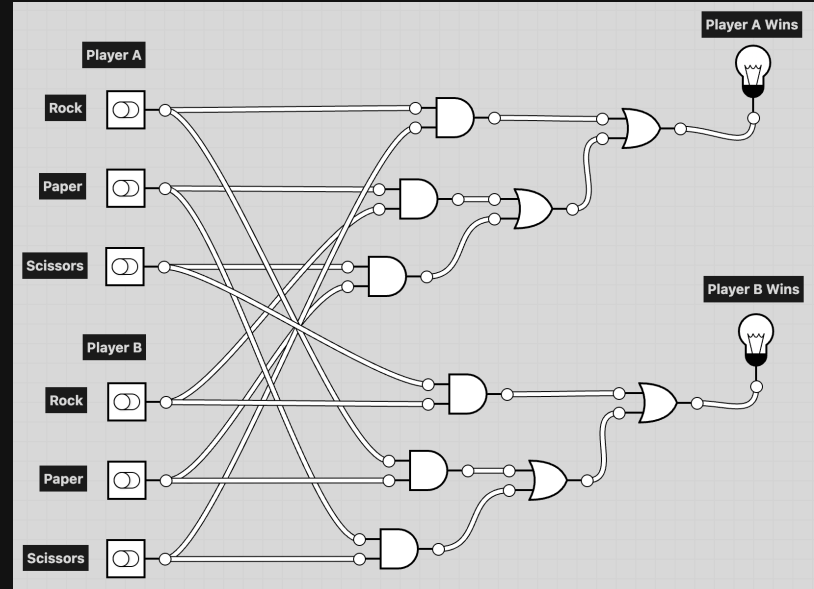


Rock, Paper, Scissors Solution

R_A	P_A	S_A	R_B	P_B	S_B	W_A	W_B
0	0	1	0	0	1	0	0
0	0	1	0	1	0	1	0
0	0	1	1	0	0	0	1
0	1	0	0	0	1	0	1
0	1	0	0	1	0	0	0
0	1	0	1	0	0	1	0
1	0	0	0	0	1	1	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0

$$W_A = S_A \cdot P_B + P_A \cdot R_B + R_A \cdot S_B$$

$$W_B = S_B \cdot P_A + P_B \cdot R_A + R_B \cdot S_A$$



Base-10 Numeral System

- 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Each "place" is the next power of 10
- $123_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$

10^3 1000	10^2 100	10^1 10	10^0 1	
0	0	0	1	$= 1 \times 10^0 = 1_{10}$
0	0	1	0	$= 1 \times 10^1 = 10_{10}$
0	1	0	0	$= 1 \times 10^2 = 100_{10}$
1	0	0	0	$= 1 \times 10^3 = 1000_{10}$
2	0	0	0	$= 2 \times 10^3 = 2000_{10}$
2	0	0	1	$= 2 \times 10^3 + 1 \times 10^0 = 2001_{10}$
4	3	2	1	$= 4 \times 10^3 + 3 \times 10^2 + 2 \times 10^1 + 1 \times 10^0 = 4321_{10}$
0	9	9	9	$= 0 \times 10^3 + 9 \times 10^2 + 9 \times 10^1 + 9 \times 10^0 = 999_{10}$
1	0	0	0	$= 1 \times 10^3 + 0 \times 10^2 + 0 \times 10^1 + 0 \times 10^0 = 1000_{10}$



Base-2 (Binary) Numeral System

- 2 digits: 0, 1
- Each "place" is the next power of 2
- $101_2 = 1 \times 2^2 + 1 \times 2^0 = 5_{10}$

2^3 8	2^2 4	2^1 2	2^0 1	
0	0	0	1	$= 1 \times 2^0 = 1_{10}$
0	0	1	0	$= 1 \times 2^1 = 2_{10}$
0	1	0	0	$= 1 \times 2^2 = 4_{10}$
1	0	0	0	$= 1 \times 2^3 = 8_{10}$
1	0	0	1	$= 1 \times 2^3 + 1 \times 2^0 = 9_{10}$
0	1	1	0	$= 1 \times 2^2 + 1 \times 2^1 = 6_{10}$
0	1	1	1	$= 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 7_{10}$
1	0	0	0	$= 1 \times 2^3 = 8_{10}$

Base-10 Addition, Example 1

$$\begin{array}{r} ^1 3 \ 5 \ 1 \\ + 1 \ 8 \ 3 \\ \hline 5 \ 3 \ 4 \end{array}$$

Base-10 Addition, Example 2

$$\begin{array}{r} ^1 9 ^1 9 9 \\ + 9 9 9 \\ \hline 1 9 9 8 \end{array}$$



Binary Addition, Example 1

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array}$$



Binary Addition, Example 2

$$\begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array}$$



Binary Addition, Example 3

$$\begin{array}{r} 1 \\ + 1 \\ \hline 10 \end{array}$$

Binary Addition, Example 4

$$\begin{array}{r} 10 \\ + 01 \\ \hline 11 \end{array} = 3_2$$

Binary Addition, Example 5

$$\begin{array}{r} ^1 1 1 \\ + 0 1 \\ \hline 1 0 \end{array} = 4_2$$

Binary Addition, Example 6

$$\begin{array}{r} 1^1 0 1 0 \\ + 0 0 1 1 \\ \hline 1 1 0 1 = 13_2 \end{array}$$



Binary Addition Truth Table

A	B	S	C _{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = A \oplus B$$

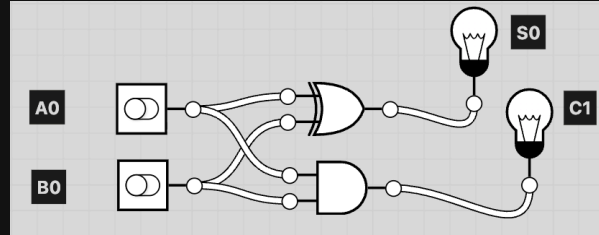
$$C_{\text{out}} = A \cdot B$$



Binary Addition with Carry Truth Table

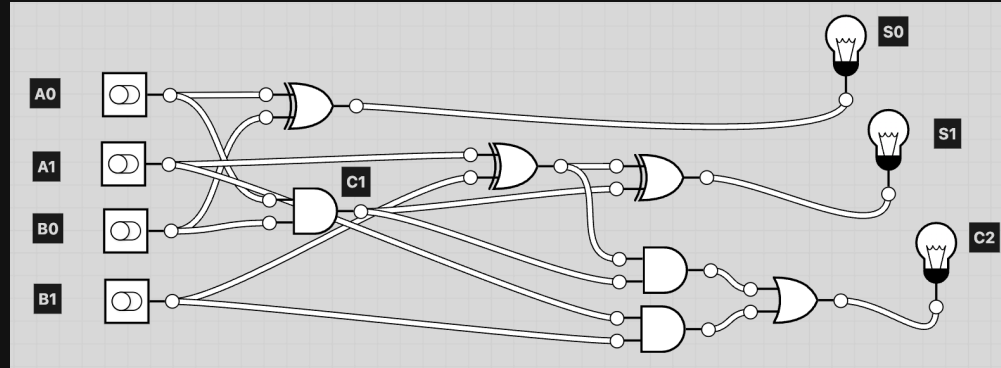
C_{in}	A	B	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

One-Bit Adder



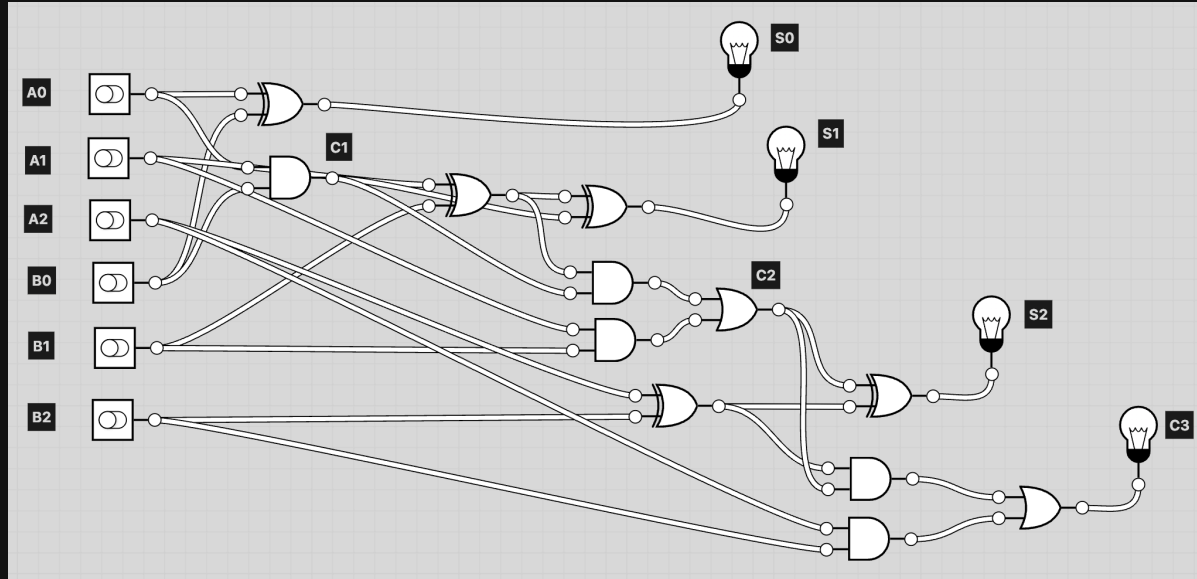


Two-Bit Adder



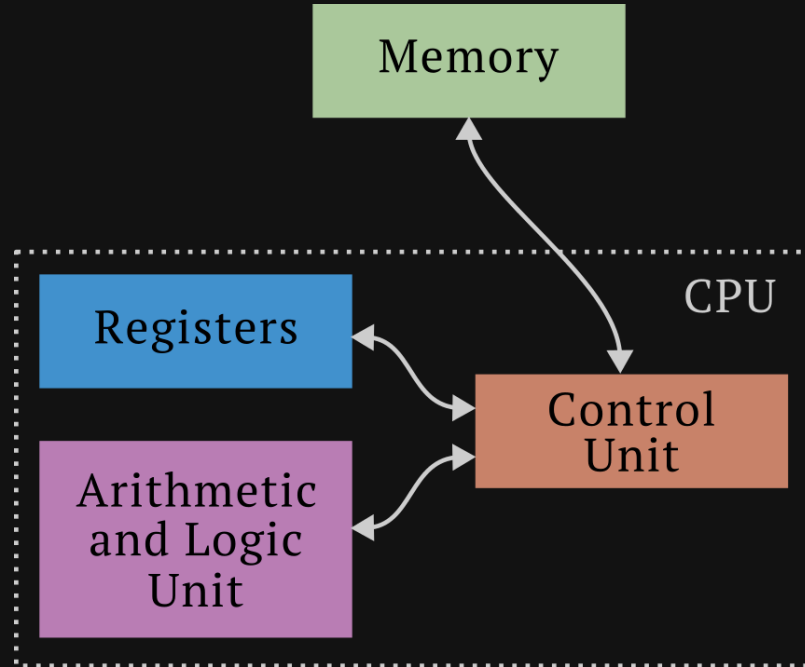


Three-Bit Adder





Computer Architecture





Base-16 (Hexadecimal) Numeral System

- 16 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- Each "place" is the next power of 16
- $1C_{16} = 1 \times 16^1 + 12 \times 16^0 = 28_{10}$

16^1	16^0
16	1

0	1	$= 1 \times 16^0 = 1_{10}$
---	---	----------------------------

0	C	$= 12 \times 16^0 = 12_{10}$
---	---	------------------------------

C	0	$= 12 \times 16^1 = 192_{10}$
---	---	-------------------------------

D	4	$= 13 \times 16^1 + 4 \times 16^0 = 212_{10}$
---	---	---

ASCII Character Encoding

- Everything in computers is stored as bits
- How do we represent characters in bits?
- Create an encoding:
 - A dictionary that maps binary values to characters
 - Most common encodings:
 - ASCII (8-bit, 1 byte)
 - UTF-8 (multi-byte unicode)

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	01	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	02	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	03	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	04	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	05	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	06	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	07	[BELL]	39	27	'	71	47	G	103	67	g
8	08	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	09	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	0A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	0B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	0C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	0D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	0E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	0F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]



Binary Numbers in Python: Literals

```
x = 5
y = 0b101 # integer literal in binary
print(f"x = {x}, y = {y}")
```

Output

```
x = 5, y = 5
```



Binary Numbers in Python: Formatting

```
y = format(x, "08b")
```

format specifier

binary

8 characters long

include leading 0's

```
x = 5
x_base_2 = format(x, "08b")
print(f"x = {x}, x_base_2 = {x_base_2}")

# Can also use same syntax inside f-string
print(f"x = {x} (base 10) = {x:08b} (base 2)")
```

Output

```
x = 5, y = 00000101
x = 5 (base 10) = 00000101 (base 2)
```

Character Encoding in Python

- `ord` converts a character to its integer value
- `chr` converts an integer to its character value

```
ch1 = "a"
x = ord(ch) # ord stands for "ordinal"
print(f'ch1 = "{ch1}" = {x}')

y = 98
ch2 = chr(y)
print(f'ch2 = "{ch2}" = {y}')
```

Output

```
ch1 = "a" = 97
ch2 = "b" = 98
```

- 97 corresponds to "a" in the ASCII table
- 98 corresponds to "b" in the ASCII table

Homework

- Write a binary encoder and decoder
- Two separate programs
- Submit both on Jupiter Ed
- Hints
 - No fancy math needed; only things like `format` , `ord` , `chr` , etc.
 - Might also need other Python functions like `input` to prompt for message and string methods like `split` . Google for them if you forgot how to use them.
 - Write the encoder first so you can use its output to test your decoder.

```
▼ TERMINAL [zsh]
(.venv) $ /Users/sjoiner/src/pyinter-2025/.venv/bin/python /Users/sjoiner/src/pyinter-2025/
assignments/assignment-04-encoder-solution.py
Enter a message: Hello!
Your message in binary is: 01001000 01100101 01101100 01101100 01101111 00100001
(.venv) $
```

```
▼ TERMINAL [zsh]
(.venv) $ /Users/sjoiner/src/pyinter-2025/.venv/bin/python /Users/sjoiner/src/pyinter-2025/
assignments/assignment-04-decoder-solution.py
Enter a binary message: 01001000 01100101 01101100 01101100 01101111 00100001
The message is: Hello!
(.venv) $
```